Data Science — laboratorium

NumPy: obsługa tablic i wektorów — część II

Cel zajęć

W pierwszej części poznaliśmy obiekt `ndarray`, jego wymiary, `shape`, `dtype`, tworzenie tablic, operacje wykonywane na całych tablicach oraz podstawowe indeksowanie i wycinanie.

Teraz wykorzystamy te podstawy do **przetwarzania danych za pomocą NumPy**.

W tej części zajmiemy się:

- indeksowaniem logicznym i maskami,
- indeksowaniem specjalnym,
- zmianą kształtu i transpozycją tablic,
- generowaniem liczb pseudolosowych,
- funkcjami uniwersalnymi NumPy,
- `np.where()`,
- metodami statystycznymi i parametrem `axis`,
- tablicami logicznymi,
- sortowaniem i wartościami unikalnymi,
- zapisem i odczytem tablic,
- podstawami algebry liniowej,
- prostą symulacją błądzenia losowego.

Każdy przykład wpisuj samodzielnie w **Google Colab**. Najpierw spróbuj przewidzieć wynik, a dopiero później uruchom komórkę.

1. Indeksowanie logiczne — wybieramy dane spełniające warunek

Utwórz tablicę:

```
temperatures = np.array([
    18.5, 21.2, 17.8, 24.3, 26.1, 19.7, 23.5
])
```

Sprawdź:

```
temperatures > 20
```

Obserwacja

Nie otrzymujemy temperatur, lecz tablicę wartości:

```
False, True, False, True, True, False, True
```

Jest to **maska logiczna**.

Możemy wykorzystać ją jako indeks:

```
temperatures[temperatures > 20]
```

W wyniku otrzymamy tylko wartości spełniające warunek.

Indeksowanie za pomocą tablic logicznych jest jednym z podstawowych sposobów wybierania danych z tablic NumPy.

Małe zadanie

Dla:

```
scores = np.array([45, 72, 88, 51, 90, 63, 39])
```

wyświetl:

1. wyniki większe lub równe `60`,
2. wyniki mniejsze niż `50`.

2. Łączenie warunków

Utwórz:

```
x = np.array([5, 12, 18, 24, 31, 42])
```

Chcemy wybrać wartości większe od `10`, ale mniejsze od `30`.

```
mask = (x > 10) & (x < 30)
print(mask)
print(x[mask])
```

Uwaga

W przypadku tablic NumPy używamy:

```
&     AND
|     OR
~     NOT
```

a nie zwykłych słów `and`, `or`, `not`.

Spróbuj:

```
x[(x < 10) | (x > 30)]
```

oraz:

```
x[~(x > 20)]
```

Małe zadanie

Dla:

```
ages = np.array([17, 19, 22, 35, 16, 28, 41])
```

wyświetl osoby:

- pełnoletnie,
- w wieku od `18` do `30` lat,
- mające mniej niż `18` lub więcej niż `35` lat.

---

3. Modyfikowanie danych za pomocą maski

Maska może służyć nie tylko do odczytywania wartości.

```
data = np.array([5, -2, 8, -7, 3, -1])

data[data < 0] = 0

print(data)
```

Obserwacja

Wszystkie wartości ujemne zostały zastąpione zerami.

Możemy w ten sposób zmieniać wiele elementów naraz bez używania pętli.

Małe zadanie

Dla:

```
scores = np.array([105, 87, -5, 64, 120, 91])
```

przyjmijmy, że poprawny wynik musi należeć do zakresu `0–100`.

Zamień:

- wartości mniejsze od `0` na `0`,
- wartości większe od `100` na `100`.

4. Indeksowanie specjalne — fancy indexing

Utwórz tablicę:

```
data = np.array([10, 20, 30, 40, 50, 60])
```

Możemy przekazać kilka indeksów jednocześnie:

```
data[[0, 2, 5]]
```

Otrzymamy elementy o indeksach `0`, `2` i `5`.

Spróbuj zmienić kolejność:

```
data[[5, 1, 3]]
```

Indeksowanie specjalne pozwala wybierać dowolne elementy tablicy w określonej kolejności.

Tablica dwuwymiarowa

```
data = np.arange(20).reshape(5, 4)

print(data)
```

Wybierz wiersze:

```
data[[0, 3, 4]]
```

Możemy nawet zmienić ich kolejność:

```
data[[4, 0, 2]]
```

Małe zadanie

Z tablicy:

```
data = np.arange(30).reshape(6, 5)
```

wyświetl kolejno:

- pierwszy wiersz,
- czwarty wiersz,
- drugi wiersz.

Zrób to **jednym wyrażeniem**.

5. reshape — zmieniamy kształt tablicy

Utwórz:

```
a = np.arange(12)

print(a)
print(a.shape)
```

Teraz:

```
b = a.reshape(3, 4)

print(b)
print(b.shape)
```

Te same dwanaście elementów zostało ułożonych jako:

```
3 wiersze × 4 kolumny
```

Spróbuj:

```
a.reshape(2, 6)
```

oraz:

```
a.reshape(4, 3)
```

Małe zadanie

Utwórz:

```
x = np.arange(24)
```

Przekształć ją kolejno do tablic:

- `4 × 6`,
- `6 × 4`,
- `3 × 8`,
- `2 × 3 × 4`.

Sprawdzaj `shape`.

6. Transpozycja tablicy

Utwórz:

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

Wyświetl:

```
a
```

a następnie:

```
a.T
```

Obserwacja

Tablica o kształcie:

```
(2, 3)
```

po transpozycji ma kształt:

```
(3, 2)
```

Wiersze stają się kolumnami, a kolumny wierszami.

Transpozycja w NumPy jest widokiem danych i jest szczególnie istotna podczas operacji macierzowych.

Małe zadanie

Utwórz:

```
x = np.arange(12).reshape(3, 4)
```

Wyświetl:

```
x.shape
x.T
x.T.shape
```

7. Liczby pseudolosowe

NumPy posiada własny generator liczb pseudolosowych.

Utwórz generator:

```
rng = np.random.default_rng(12345)
```

Wygeneruj pięć liczb:

```
rng.random(5)
```

Liczby całkowite od `1` do `6`:

```
rng.integers(1, 7, size=10)
```

Możemy więc zasymulować dziesięć rzutów kostką.

Rozkład normalny:

```
rng.standard_normal(5)
```

McKinney przedstawia nowoczesny interfejs oparty na obiekcie generatora utworzonym przez `np.random.default_rng()`.

Małe zadanie

Za pomocą `rng`:

1. zasymuluj dwadzieścia rzutów kostką,
2. wygeneruj tablicę `3 × 4` liczb z rozkładu normalnego,
3. wygeneruj dziesięć liczb całkowitych z zakresu `0–99`.

---

8. Dlaczego ustawiamy ziarno generatora?

Porównaj:

```
rng1 = np.random.default_rng(123)
rng2 = np.random.default_rng(123)

print(rng1.integers(0, 100, 5))
print(rng2.integers(0, 100, 5))
```

Obserwacja

Oba generatory dają tę samą sekwencję.

Dzięki ustawieniu ziarna możemy **powtórzyć eksperyment**.

W analizie danych i uczeniu maszynowym jest to bardzo ważne dla odtwarzalności wyników.

Małe zadanie

Zmień ziarno drugiego generatora na:

```
456
```

Porównaj wyniki.

---

9. Funkcje uniwersalne — ufunc

NumPy zawiera funkcje wykonujące operację na każdym elemencie tablicy.

Utwórz:

```
x = np.array([1, 4, 9, 16, 25])
```

Pierwiastek:

```
np.sqrt(x)
```

Potęgowanie:

```
np.square(x)
```

Wartość bezwzględna:

```
y = np.array([-5, -2, 0, 3, 7])
```

```
np.abs(y)
```

Funkcja wykładnicza:

```
np.exp(np.array([0, 1, 2]))
```

Funkcje uniwersalne działają element po elemencie na całych tablicach. McKinney wyróżnia funkcje jednoargumentowe i dwuargumentowe.

10. Funkcje działające na dwóch tablicach

Utwórz:

```
a = np.array([3, 8, 2, 7])
b = np.array([5, 4, 9, 1])
```

Sprawdź:

```
np.maximum(a, b)
```

NumPy porównuje odpowiadające sobie elementy:

```
3 z 5
8 z 4
2 z 9
7 z 1
```

i wybiera większy.

Spróbuj również:

```
np.minimum(a, b)
```

Małe zadanie

Dane są wyniki dwóch pomiarów:

```
measurement_a = np.array([12.1, 15.4, 9.8, 17.0])
measurement_b = np.array([11.9, 16.2, 10.1, 15.8])
```

Znajdź:

- większy wynik dla każdej pary,
- mniejszy wynik dla każdej pary.

---

11. np.where() — warunek bez pętli

Utwórz:

```
x = np.array([-4, 2, -1, 5, 8, -3])
```

Chcemy zastąpić liczby dodatnie wartością `1`, a pozostałe `0`.

```
result = np.where(x > 0, 1, 0)

print(result)
```

Schemat:

```
np.where(warunek, wartość_gdy_True, wartość_gdy_False)
```

`np.where()` jest przykładem zastępowania warunkowego wykonywanego bezpośrednio na całej tablicy.

Małe zadanie

Dla:

```
temperatures = np.array([18, 24, 31, 27, 35, 20])
```

utwórz tablicę napisów:

```
"gorąco"
```

dla temperatur powyżej `30`, a:

```
"OK"
```

dla pozostałych.

---

12. Podstawowe statystyki

Utwórz:

```
data = np.array([10, 12, 14, 18, 21, 25])
```

Sprawdź:

```
data.sum()
data.mean()
data.min()
data.max()
data.std()
```

Możemy również znaleźć indeks najmniejszej i największej wartości:

```
data.argmin()
data.argmax()
```

Małe zadanie

Dane są wyniki testu:

```
scores = np.array([62, 75, 81, 90, 55, 88, 71])
```

Oblicz:

- średnią,
- minimum,
- maksimum,
- odchylenie standardowe,
- indeks najlepszego wyniku.

---

13. axis — obliczenia w wierszach i kolumnach

Utwórz:

```
data = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])
```

Średnia wszystkich elementów:

```
data.mean()
```

Teraz:

```
data.mean(axis=0)
```

oraz:

```
data.mean(axis=1)
```

Obserwacja

Dla tablicy dwuwymiarowej:

```
axis=0  → redukujemy wiersze → wynik dla każdej kolumny
axis=1  → redukujemy kolumny → wynik dla każdego wiersza
```

Metody statystyczne NumPy mogą agregować całą tablicę albo działać wzdłuż wybranej osi.

Małe zadanie

Dla tablicy `data` policz:

- sumę każdej kolumny,
- maksimum każdej kolumny,
- średnią każdego wiersza.

14. Suma skumulowana

Utwórz:

```
daily_sales = np.array([10, 15, 12, 20, 18])
```

Sprawdź:

```
daily_sales.cumsum()
```

Obserwacja

Otrzymamy kolejno:

```
sprzedaż po dniu 1
sprzedaż po dniach 1–2
sprzedaż po dniach 1–3
...
```

`cumsum()` będzie bardzo przydatne również w końcowym zadaniu z błądzeniem losowym.

---

15. Tablice logiczne: sum(), any(), all()

Utwórz:

```
data = np.array([12, -3, 8, -1, 5, 10])
```

Ile wartości jest dodatnich?

```
(data > 0).sum()
```

Czy istnieje wartość ujemna?

```
(data < 0).any()
```

Czy wszystkie wartości są dodatnie?

```
(data > 0).all()
```

Małe zadanie

Dla:

```
scores = np.array([75, 82, 91, 68, 77])
```

sprawdź:

1. ile wyników jest większych lub równych `75`,
2. czy istnieje wynik większy niż `90`,
3. czy wszystkie wyniki są większe niż `50`.

Metody `any()` i `all()` są podstawowymi operacjami redukującymi tablice logiczne do pojedynczej wartości.

---

16. Sortowanie

Utwórz:

```
data = np.array([7, 2, 9, 1, 5])
```

Spróbuj:

```
np.sort(data)
```

Następnie:

```
print(data)
```

Obserwacja

`np.sort()` zwraca posortowaną kopię.

Możemy również wykonać:

```
data.sort()
```

i ponownie:

```
print(data)
```

Tym razem zmieniła się oryginalna tablica.

McKinney zwraca uwagę na różnicę między metodą `sort()` modyfikującą tablicę a funkcją `np.sort()` zwracającą posortowaną kopię.

Małe zadanie

Posortuj:

```
scores = np.array([81, 63, 95, 72, 88, 54])
```

tak, aby oryginalna tablica pozostała niezmieniona.

17. Wartości unikalne

Utwórz:

```
animals = np.array([
    "kot", "pies", "kot", "lama",
    "pies", "lama", "kot"
])
```

Uruchom:

```
np.unique(animals)
```

Obserwacja

Otrzymujemy każdą wartość tylko raz.

Spróbuj:

```
values, counts = np.unique(animals, return_counts=True)

print(values)
print(counts)
```

Małe zadanie

Dla:

```
grades = np.array([5, 4, 3, 5, 4, 5, 2, 3, 4, 5])
```

sprawdź:

- jakie różne oceny wystąpiły,
- ile razy wystąpiła każda ocena.

---

18. Działania na zbiorach

Utwórz:

```
a = np.array([1, 2, 3, 4, 5])
b = np.array([4, 5, 6, 7])
```

Część wspólna:

```
np.intersect1d(a, b)
```

Suma zbiorów:

```
np.union1d(a, b)
```

Sprawdzenie obecności:

```
np.isin(a, b)
```

Operacje na wartościach unikalnych i zbiorach stanowią osobną grupę narzędzi NumPy omawianą przez McKinneya.

Małe zadanie

Mamy identyfikatory studentów:

```
group_a = np.array([101, 102, 103, 104])
group_b = np.array([103, 104, 105, 106])
```

Znajdź:

- studentów należących do obu grup,
- wszystkie różne identyfikatory występujące w obu grupach.

---

19. Zapis tablicy do pliku

Utwórz:

```
temperatures = np.array([18.5, 20.1, 19.8, 22.4])
```

Zapisz:

```
np.save("temperatures.npy", temperatures)
```

Wczytaj:

```
loaded = np.load("temperatures.npy")

print(loaded)
```

Format `.npy` pozwala przechować pojedynczą tablicę NumPy.

Kilka tablic można zapisać w jednym archiwum:

```
a = np.arange(5)
b = np.arange(10, 15)

np.savez("data.npz", first=a, second=b)
```

Odczyt:

```
archive = np.load("data.npz")

print(archive["first"])
print(archive["second"])
```

NumPy udostępnia własne binarne formaty `.npy` i `.npz` służące do szybkiego zapisywania i odczytywania tablic.

20. Podstawy algebry liniowej

Utwórz dwie macierze:

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])
```

Przypomnijmy:

```
A * B
```

wykonuje mnożenie **element po elemencie**.

Mnożenie macierzy wykonujemy:

```
A @ B
```

Możemy również użyć:

```
np.dot(A, B)
```

Operacje algebry liniowej, w tym iloczyn macierzowy, są częścią narzędzi NumPy przedstawionych w dalszej części rozdziału.

Małe zadanie

Spróbuj przewidzieć różnicę między:

```
A * B
```

a:

```
A @ B
```

Następnie porównaj wyniki.

21. Rozwiązywanie prostego układu równań

Załóżmy układ:

```
2x + y = 8
x + 3y = 9
```

Możemy zapisać:

```
A = np.array([
    [2, 1],
    [1, 3]
])

b = np.array([8, 9])
```

Rozwiązanie:

```
solution = np.linalg.solve(A, b)

print(solution)
```

Sprawdź wynik:

```
A @ solution
```

Powinniśmy odzyskać:

```
b
```

Małe zadanie

Rozwiąż:

```
x + y = 5
2x + 3y = 12
```

za pomocą `np.linalg.solve()`.

---

22. Finał — błądzenie losowe

Na końcu połączymy kilka poznanych elementów.

Wyobraźmy sobie osobę stojącą w punkcie `0`.

W każdym kroku może przesunąć się:

```
+1  → krok w prawo
-1  → krok w lewo
```

Utwórz generator:

```
rng = np.random.default_rng(12345)
```

Wygeneruj sto losowych kroków:

```
steps = rng.choice([-1, 1], size=100)

print(steps)
```

Teraz policz kolejne pozycje:

```
walk = steps.cumsum()

print(walk)
```

Obserwacja

`cumsum()` zamienił pojedyncze kroki w kolejne pozycje spacerowicza.

Przykładowo:

```
kroki:     1   1  -1   1  -1
pozycja:   1   2   1   2   1
```

Błądzenie losowe jest przykładem zamykającym rozdział McKinneya i pokazuje, jak operacje tablicowe mogą zastąpić pętle przy symulacji wielu kolejnych zdarzeń.

---

23. Zbadajmy nasze błądzenie

Najwyższa osiągnięta pozycja:

```
walk.max()
```

Najniższa:

```
walk.min()
```

Końcowa pozycja:

```
walk[-1]
```

Ile razy byliśmy po dodatniej stronie?

```
(walk > 0).sum()
```

Czy spacerowicz kiedykolwiek znalazł się co najmniej dziesięć kroków od punktu startu?

```
(np.abs(walk) >= 10).any()
```

Małe zadanie

Zmień liczbę kroków:

```
size=1000
```

i sprawdź:

1. pozycję końcową,
2. pozycję minimalną,
3. pozycję maksymalną,
4. ile razy spacerowicz znajdował się po dodatniej stronie,
5. czy kiedykolwiek oddalił się od początku o co najmniej `20`.

---

24. Pierwsze osiągnięcie

Wykonaj:

```
mask = np.abs(walk) >= 20
```

Sprawdź:

```
mask.any()
```

Jeżeli wynikiem jest `True`, możemy znaleźć pierwszy moment osiągnięcia tej odległości:

```
first = mask.argmax()

print(first)
print(walk[first])
```

Obserwacja

Połączyliśmy tutaj:

- liczby losowe,
- `cumsum()`,
- wartość bezwzględną,
- maskę logiczną,
- `any()`,
- `argmax()`.

To już bardzo typowy sposób pracy z NumPy: **zamiast analizować każdy element osobno, tworzymy wyrażenie działające na całej tablicy**.

25. Zadanie końcowe — mały eksperyment

Zasymuluj `1000` losowych kroków:

```
rng = np.random.default_rng(2026)

steps = rng.choice([-1, 1], size=1000)
walk = steps.cumsum()
```

Następnie samodzielnie oblicz:

1. końcową pozycję spacerowicza,
2. minimalną i maksymalną pozycję,
3. średnią pozycję,
4. odchylenie standardowe pozycji,
5. ile razy spacerowicz znajdował się powyżej zera,
6. ile razy znajdował się poniżej zera,
7. czy osiągnął odległość co najmniej `25` od punktu startowego,
8. w którym kroku po raz pierwszy osiągnął tę odległość,
9. dziesięć największych wartości `walk`,
10. ile różnych pozycji odwiedził.

Dodatkowe wyzwanie

Spróbuj wykonać wszystkie obliczenia **bez ani jednej pętli `for`**.

Po zakończeniu części II

Powinieneś potrafić:

- tworzyć i wykorzystywać maski logiczne,
- łączyć kilka warunków,
- wybierać elementy za pomocą fancy indexing,
- zmieniać kształt tablic przez `reshape()`,
- transponować tablice,
- generować dane pseudolosowe,
- korzystać z funkcji uniwersalnych,
- używać `np.where()`,
- obliczać podstawowe statystyki,
- rozumieć działanie parametru `axis`,
- używać `sum()`, `any()` i `all()` dla warunków logicznych,
- sortować tablice,
- znajdować wartości unikalne,
- wykonywać podstawowe operacje zbiorowe,
- zapisywać i odczytywać tablice `.npy` i `.npz`,
- odróżniać mnożenie element po elemencie od mnożenia macierzowego,
- używać podstawowych funkcji `np.linalg`,
- przeprowadzić prostą symulację opartą na operacjach wektorowych.

Najważniejsza idea po obu częściach

W zwykłym Pythonie często myślimy:

`„weź kolejny element → wykonaj działanie → przejdź do następnego”`

W NumPy staramy się myśleć:

`„wykonaj działanie na całej tablicy”`

To właśnie **wektoryzacja** będzie później jednym z fundamentów pracy z danymi w NumPy, Pandas i wielu narzędziach Data Science.